d/Senior Software Engineer

1. Requirements Analysis

Your diagram captures most core requirements. Below is a structured breakdown with interview-ready additions.

1.1 Functional Requirements

In Your Design ✓	Gaps to Add ✗
• Retrieve ride types and fare estimates (src, dest)	• **[Missing]** Real-time ride tracking for customer
• Book a ride (userId, rideTypeId, fare)	• **[Missing]** Driver goes online / offline toggle
• Cancel a ride (rider **or** driver)	• **[Missing]** Surge pricing / dynamic fare adjustment
• Driver accepts / declines ride request	• **[Missing]** Payment processing at trip end
• OTP verification at ride start	• **[Missing]** Trip history for rider and driver
• In-app call and messaging	• **[Missing]** Driver earnings summary
• Rating and review after trip	• **[Missing]** Admin / ops dashboard (cancellations, fraud)

1.2 Non-Functional Requirements

> **Your design correctly chose AP (Availability + Partition Tolerance)** per CAP theorem. Uber operates globally; network partitions are inevitable. Eventual consistency is acceptable for most operations (driver location, fare display), but **ride booking confirmation must be linearisable** — exactly-once semantics needed so a driver is not double-booked.

Availability	99.99% uptime target — multi-AZ, active-active deployment across regions.
Scalability	Millions of concurrent ride requests during peak hours (NYE, rain surge). Services must scale horizontally; stateless microservices behind a load balancer.
Low-latency matching	<500 ms to present matched driver to rider. GeoHash + Redis in-memory lookup critical.
1:1 Driver–Rider match	Distributed lock (Redis TTL) ensures no two requests are dispatched to same driver simultaneously.
[Missing] Consistency for payments	Payment deduction must be ACID — consider a separate SQL DB (PostgreSQL/Aurora) for financial transactions.
[Missing] Location update throughput	~1 GPS ping every 4–5 s per active driver. At 500k active drivers = 100–125k writes/sec. Kafka + dedicated write path required (captured in your design ✓).
[Missing] Data freshness SLA	Driver location in GeoHash cache must be <10 s stale. TTL-based eviction in Redis needed.

2. API Design

Your API surface is good. Below is an enhanced version with HTTP semantics, auth headers, response shapes, and the missing endpoints an interviewer will probe.

2.1 Existing APIs – Enhanced

`GET /v1/rides/options`

```
Headers: Authorization: Bearer <JWT>
Body: { "src": "lat,lng", "dest": "lat,lng" }
Response: 200 OK: [{ rideTypeId, label, eta_min, fare_range }] | 400 / 401
```

`POST /v1/rides`

```
Headers: Authorization: Bearer <JWT> X-Idempotency-Key: <uuid>
Body: { "userId": "u123", "rideTypeId": "UX", "fareToken": "tok_abc" }
Response: 202 Accepted: { rideId, status:"SEARCHING" } | 409 if duplicate key
```

`POST /v1/rides/{rideId}/cancel`

```
Headers: Authorization: Bearer <JWT>
Body: { "reason": "changed_mind" }
Response: 200 OK: { rideId, status:"CANCELLED", cancellation_fee }
```

`POST /v1/rides/{rideId}/review`

```
Headers: Authorization: Bearer <JWT>
Body: { "driverId": "d456", "rating": 5, "comment": "..." }
Response: 201 Created: { reviewId }
```

2.2 Missing APIs

> **These will almost certainly be asked about in an interview:**

PATCH /v1/drivers/{driverId}/status	Driver goes online/offline. Body: { status: 'ONLINE'|'OFFLINE', location: 'lat,lng' }
POST /v1/drivers/location	Batch GPS heartbeat. Body: { driverId, lat, lng, timestamp }. High-throughput; Kafka-bound.
POST /v1/rides/{rideId}/otp/verify	Driver submits OTP to start trip. Body: { otp: '1234' }. Response: { status: 'RIDE_STARTED' }
POST /v1/rides/{rideId}/complete	Driver marks trip complete. Triggers payment flow.
GET /v1/rides/{rideId}/status	Long-poll or SSE endpoint for real-time ride status (SEARCHING, MATCHED, EN_ROUTE, ARRIVED, COMPLETED).
GET /v1/riders/{riderId}/trips	Paginated trip history. Query params: page, limit, from_date, to_date.

3. Architecture & Request Flow

Your diagram has a numbered flow. Below is a clarified and extended walkthrough with the gaps filled in.

3.1 Booking Flow (Steps 1–17)

1	**User submits ride options**	Customer/Driver apps → API Gateway → RideType & Fare Service. Source + destination sent.
2	**ETA retrieval**	RideType & Fare Service calls Google Maps / external routing API (dashed line in diagram) to get ETA.
3	**Fare recommendation**	ML Fare Recommendation System (blue box in diagram) calculates fare from src, dest, rideType, ETA, demand surge.
4	**Response to user**	Fare + ETA returned synchronously to user. **Interview tip:** mention this could be async via webhook for resilience.
5	**Booking pushed to queue**	User selects a fare and confirms → POST /v1/rides → API Gateway pushes event to Kafka queue (high throughput decoupling).
6	**Book Ride Service consumes**	Book Ride Service is a Kafka consumer. Processes booking, writes to DynamoDB.
7	**Persist as RIDE_REQUESTED**	Entry saved with status=RIDE_REQUESTED, driverId=null. OTP generated and stored.
8	**GeoHash lookup**	Query Redis GeoHash store for drivers within radius (e.g. 2 km). Returns sorted list by proximity.
9	**Distributed driver lock**	Iterate driver list. For each driver: acquire Redis lock (TTL=5s). If lock acquired, send push notification.
10	**Notification to driver**	Webhook / push notification (FCM/APNS) pops on driver app screen.
11	**Driver responds**	Driver accepts or declines → POST /v1/rides/{id}/accept → Ride Confirmation Service (synchronous).
12	**Status update**	On accept: update DynamoDB status=BOOKING_CONFIRMED, driverId=driver. Release lock. Break loop.
13	**Driver GPS heartbeat**	Driver app POSTs location every ~4s → Kafka topic: driver-location-updates.
14–15	**Location update service**	Consumes Kafka events → updates Redis GeoHash store. Handles 100k+ writes/sec.
16	**OTP verification**	At pickup: driver enters OTP → OTP Verification Service checks against value stored in DynamoDB → RIDE_STARTED.
17	**OTP stored in DB**	OTP value flows from Book Ride Service → DynamoDB (confirmed in diagram).

3.2 Missing Flows

✗ **Payment flow**	After driver marks COMPLETED, a Payment Service should deduct fare from rider's stored card (Stripe/Braintree), credit driver earnings, and record a payment ledger entry in a SQL DB (ACID required).
✗ **Real-time tracking**	Once BOOKING_CONFIRMED, rider needs live driver location. Use WebSocket or Server-Sent Events (SSE) from a Location Stream Service that reads the Redis GeoHash and pushes delta updates every few seconds.
✗ **Surge pricing signal**	Demand/supply ratio must be computed continuously. A Surge Pricing Service (or the Fare ML system) should subscribe to ride-request volume metrics and update a dynamic multiplier in Redis (fast read by all services).
✗ **Cancellation & timeout**	If no driver accepts after N attempts (e.g. all drivers in 5 km declined), the system must gracefully cancel the ride and notify the user. Add a timeout job (e.g. Quartz / DynamoDB TTL trigger).
✗ **Driver earnings ledger**	Post-trip, credit the driver's wallet. This needs an append-only ledger table — strongly consistent SQL write (no NoSQL here).

4. Technology Stack – Justification & Interview Critique

Your design uses DynamoDB, Redis, and Kafka. Here is a rigorous justification for each, counterarguments to expect, and the cases where a different choice would be better.

DynamoDB	**Ride bookings, driver & rider profiles**	• Single-digit ms reads at any scale. • No schema migration downtime — rider/driver profile schema evolves independently. • Global Tables for multi-region replication. • DynamoDB Streams can trigger Lambda for event-driven workflows (e.g. OTP expiry). **Partition key choice:** rideId (UUID) distributes evenly. GSI on riderId + createdAt for trip history queries. **Interviewer pushback:** No ACID! — counter: booking confirmation uses conditional writes (optimistic locking on version attribute). For payment, you switch to Aurora/PostgreSQL (ACID).
Redis (GeoHash)	**Driver proximity lookup**	• GEOADD / GEORADIUS commands — O(N+log(M)) for proximity search. • Sub-millisecond reads — critical for <500 ms matching SLA. • Set TTL per driver key so stale offline drivers are auto-evicted. **Alternative:** PostGIS (PostgreSQL spatial extension) — better for complex polygon queries (geofencing) but higher latency and operational overhead than Redis at this scale. **Interviewer pushback:** What if Redis goes down? — use Redis Sentinel / Redis Cluster with replication. Fallback: re-populate from last known location in DynamoDB.
Redis (Distributed Lock)	**Prevent double-dispatch to driver**	• SETNX + EXPIRE provides a simple, low-latency mutex. • Redlock algorithm across 3+ Redis nodes for stronger guarantees. • TTL (5 s) auto-releases if Book Ride Service crashes mid-loop. **Interviewer pushback:** Clock skew makes Redlock unsafe. Acknowledge the Martin Kleppmann critique; in practice Uber's scale demands pragmatic distributed locks, and idempotency checks on the DB side provide a safety net.

Kafka	**Booking queue & location updates**	• Durable, replayable log — critical for auditing and re-processing failed bookings. • High throughput: handles millions of GPS pings/sec with appropriate partition count. • Consumer groups: Book Ride Service, Notification Service, Analytics can all consume independently. **Topic design:** ride-booking-requests (keyed by userId for ordering), driver-location-updates (keyed by driverId for ordered per-driver updates). **Interviewer pushback:** Why not SQS? — SQS lacks replay and consumer-group semantics. For location updates volume, Kafka partitioning scales better. **[Missing]** Dead-letter queue for failed booking events. Add DLQ with alerting.
API Gateway	**Auth, rate limiting, routing**	• Single ingress point — JWT validation, rate limiting (e.g. 60 req/min per user), and routing to downstream microservices. • TLS termination here; all internal traffic can be mTLS. **[Missing]** Mention WebSocket upgrade at the gateway for real-time location streaming. **[Missing]** Circuit breaker (Hystrix / Resilience4j) at the gateway to gracefully degrade.
[Missing] PostgreSQL/Aurora	**Payments & driver earnings**	Payments require ACID guarantees. NoSQL is the wrong choice here. • Use Aurora PostgreSQL (managed, read replicas, point-in-time recovery). • Tables: payments(paymentId, rideId, amount, status, createdAt), driver_earnings(driverId, period, gross, deductions, net). • Saga pattern or Two-Phase Commit for distributed transaction (charge rider, credit driver).

5. Database Schema Design

Your diagram mentions the key tables. Here they are fleshed out with field types, partition keys, and the missing tables.

5.1 DynamoDB Tables (Core)

Table	Keys & Indexes	Attributes
RideBookingRequest	PK: rideId (UUID) GSI1: riderId + createdAt (trip history) GSI2: driverId + status (driver's current ride)	rideId, riderId, driverId (nullable), src, dest, fare, rideType, status [REQUESTED|CONFIRMED|STARTED|COMPLETED|CANCELLED], otp, createdAt, updatedAt
DriverTable	PK: driverId GSI: status + city (find online drivers in city)	driverId, name, phone, vehicleId, licencePlate, rating, status [ONLINE|OFFLINE|ON_TRIP], currentLocation, city, createdAt
RiderTable	PK: riderId GSI: email	riderId, name, email, phone, defaultPaymentMethodId, rating, createdAt

5.2 Missing Tables

Table	Fields
payments **(PostgreSQL)**	paymentId, rideId, riderId, driverId, amount, currency, status [PENDING|CAPTURED|FAILED|REFUNDED], paymentMethodId, stripeChargeId, createdAt, updatedAt
driver_earnings **(PostgreSQL)**	earningId, driverId, rideId, grossFare, platformFee, netEarning, period, settledAt
LocationHistory **(DynamoDB / S3 + Athena)**	driverId + timestamp (composite PK), lat, lng, speed, heading. Hot data in DynamoDB (last 24h), cold data in S3 Parquet for analytics.
RatingsReviews **(DynamoDB)**	reviewId, rideId, reviewerId, targetId (driver or rider), rating (1-5), comment, createdAt

6. Scalability Deep Dives

6.1 Driver Matching — GeoHash vs. Alternatives

Method	Complexity	Pros	Cons
GeoHash (Redis)	O(N) range search on hash prefix	Simple, fast. Good for city-level granularity.	Not optimal near hash boundaries → use 8-neighbours search.
QuadTree	O(log N) recursive subdivision	Dynamic density adaptation.	Complex to implement; harder to shard.
S2 Geometry (Google)	Hierarchical cell decomposition	Used by Uber in production. Best spatial accuracy.	Custom library; steeper learning curve.
R-Tree (PostGIS)	O(log N) bounding box	Best for polygon/geofence queries.	DB I/O bound — too slow at 100k writes/sec.

6.2 Location Update Throughput Estimation

Back-of-envelope (must demonstrate this in interview):

• Active drivers globally (peak): ~500,000

• GPS ping frequency: every 5 s

• Writes/sec: 500,000 / 5 = **100,000 writes/sec**

• Each event: ~200 bytes → **20 MB/s Kafka throughput** (easily handled with 10–20 partitions on modern hardware)

• Redis GEOADD at 100k ops/sec: requires Redis Cluster with 5–10 shards (each shard handles ~30k ops/sec)

6.3 Ride Matching Concurrency — Distributed Lock Deep Dive

Your design correctly uses Redis for distributed locking. Here's the precise algorithm to articulate:

a	**Get candidate drivers**	Redis GEORADIUS / GEOSEARCH → sorted list of driverIds by distance.
b	**Acquire lock per driver**	SET driver:lock:{driverId} {rideId} NX PX 5000 (SET if Not eXists, expires in 5000ms).
c	**Send notification**	If lock acquired: push FCM/APNS notification to driver's device.
d	**Await response**	Wait up to 5 s for driver ACK (e.g. via WebSocket or polling).
e	**On ACK**	Driver accepted → update DB → break loop → release lock explicitly (DEL driver:lock:{driverId}).
f	**On timeout/decline**	Lock auto-expires after 5 s (TTL). Move to next driver in list.
g	**All exhausted**	If no driver accepts after N candidates → trigger RIDE_NOT_FOUND, notify rider, set status=CANCELLED.

6.4 Real-time Rider Tracking (Missing from Design)

Architecture: Long-polling vs. WebSocket vs. SSE

Recommended: **WebSocket** (bidirectional, low overhead after handshake).

• A dedicated Location Stream Service maintains WebSocket connections from rider apps.

• It subscribes to a Kafka topic (driver-location-{rideId}) published by the Location Update Service.

• Pushes location delta to the connected rider every 3–5 s.

• On COMPLETED/CANCELLED, server closes the WebSocket connection.

• Scale: Use sticky sessions (consistent hashing by rideId) so each WebSocket server handles one ride's stream.

7. Comprehensive Gap Analysis

A prioritised list of everything missing from the original design, with impact and recommended solution.

Priority	Gap	Problem	Recommended Solution
HIGH	**Payment Service**	No payment flow at all.	Add a Payment Service post-trip completion. Integrate Stripe/Braintree. Use a saga (charge rider → credit driver) with a compensation step on failure. Store in PostgreSQL for ACID.
HIGH	**Real-time location streaming to rider**	Rider has no live driver tracking.	WebSocket-based Location Stream Service. Subscribe to Kafka driver-location topic, push to rider app.
HIGH	**Driver online/offline state**	System doesn't know which drivers are available.	PATCH /v1/drivers/{id}/status endpoint. On ONLINE: GEOADD to Redis. On OFFLINE: GEOREM + update DynamoDB.
HIGH	**Cancellation timeout / no-driver scenario**	Infinite loop risk if no drivers respond.	Bounded retry loop (e.g. 10 drivers max). Timeout job after 3 min → auto-cancel + notify rider.
MEDIUM	**Surge / dynamic pricing signal**	Static fares only.	Demand signal from ride-request volume → multiplier stored in Redis → Fare ML system reads it.
MEDIUM	**Circuit breakers & retries**	No fault tolerance between services.	Add Resilience4j / Hystrix at API Gateway and inter-service calls. Exponential backoff on transient failures.
MEDIUM	**Idempotency on POST /v1/rides**	Double-tapping 'Book' may create duplicate rides.	X-Idempotency-Key header. Store key → rideId mapping in Redis (TTL 24h). Return existing rideId if duplicate key.
MEDIUM	**Trip history pagination**	No history API defined.	GET /v1/riders/{id}/trips with DynamoDB GSI on riderId + createdAt. Paginate with LastEvaluatedKey.
MEDIUM	**Notification Service abstraction**	Notifications mixed into Book Ride Service.	Extract Notification Service: consumes from Kafka, dispatches FCM/APNS/SMS. Decouples delivery from booking logic.
LOW	**Analytics / Data Warehouse**	No analytics path.	Kafka → Flink/Spark Streaming → S3 (Parquet) → Redshift/Athena for business reporting, fraud detection.
LOW	**Dead-letter queue (DLQ)**	Failed Kafka messages are silently dropped.	Configure DLQ topic. Alert on DLQ depth. Re-processing job for failed booking events.
LOW	**Admin & ops tooling**	No back-office mentioned.	Internal admin service: override ride status, investigate fraud, manage driver incentives.

8. Interview Coaching – Questions & Model Answers

Q: Why DynamoDB over PostgreSQL for ride bookings?

Model Answer: Ride volume is write-heavy (millions of bookings/day) and access patterns are simple: look up by rideId or by userId+status. DynamoDB's single-digit ms latency and horizontal auto-scaling handles this better than managing PostgreSQL read replicas. However, I'd use PostgreSQL/Aurora for the payments table because that requires multi-row ACID transactions — charging the rider and crediting the driver must be atomic.

Q: How do you handle two riders booking the same driver simultaneously?

Model Answer: Redis distributed lock (SETNX + EXPIRE). When the Book Ride Service picks a driver, it attempts to acquire a 5-second lock on that driverId. Only one request can hold the lock at a time. The other request moves to the next driver in the sorted proximity list. The TTL ensures the lock is auto-released if the service crashes.

Q: What happens if Kafka is down?

Model Answer: Kafka is the critical path for booking. Mitigations: (1) Kafka clusters with replication factor 3 — single-broker failures are transparent. (2) Producer retries with exponential backoff. (3) Circuit breaker: if Kafka is unreachable after N retries, fall back to synchronous processing (degraded mode — lower throughput but system stays up). (4) Health monitoring with PagerDuty alerts on consumer lag.

Q: How do you scale to 10x traffic during NYE?

Model Answer: Stateless microservices scale horizontally — spin up more Book Ride Service pods via Kubernetes HPA. Kafka handles burst by buffering events — consumer lag increases temporarily but no data is lost. Redis Cluster scales by adding shards. DynamoDB auto-scales read/write capacity. Pre-warm: use load testing data to pre-provision resources ahead of the event.

Q: How would you handle the OTP to prevent fraud?

Model Answer: OTP is a 4-6 digit code generated server-side at booking confirmation. Stored encrypted in DynamoDB alongside the ride. The customer sees it in-app. At pickup, the driver enters the OTP into their app, which calls POST /v1/rides/{id}/otp/verify. The service compares hashes (bcrypt or HMAC-SHA256). Rate-limit OTP attempts (max 3 per ride). OTP expires when the ride starts or after 30 minutes.

Q: What consistency model does your system use?

Model Answer: Eventual consistency for most reads (driver location, ride status display). Strong consistency (DynamoDB conditional writes) for booking state transitions (REQUESTED → CONFIRMED must be atomic — uses a version attribute for optimistic locking). Strict serialisability for payments (Aurora PostgreSQL with serialisable isolation).

Q: How do you prevent a cancelled ride from still being charged?

Model Answer: Saga pattern: the booking flow is decomposed into compensable transactions. If any step fails (e.g. payment fails), the saga orchestrator issues compensation commands (cancel ride, unlock driver, refund). Each service exposes a compensate endpoint. The saga state is persisted in DynamoDB for durability.

9. Design Strengths & Overall Assessment

9.1 What Your Design Does Well

✓ Correct CAP theorem choice — AP with eventual consistency justified.

✓ GeoHash + Redis for driver proximity — industry-standard, correct complexity awareness.

✓ Distributed Redis lock with TTL for exactly-once driver notification — shows deep understanding.

✓ Kafka decoupling of booking from processing — enables high throughput and replay.

✓ Fare Recommendation ML System included — demonstrates awareness of real-world complexity.

✓ OTP verification service included — security-minded design.

✓ Numbered flow with explanations — easy to walk an interviewer through.

✓ Separate services for each domain (RideType, Booking, Confirmation, Location Update) — good microservice decomposition.

9.2 Overall Interview Readiness

Area	Score	Notes
Requirements Coverage	**7/10**	Missing payment, tracking, driver online/offline
API Design	**6/10**	Good core endpoints; missing tracking, payment, history APIs
Architecture	**8/10**	Solid flow; missing real-time tracking & payment path
Database Choice	**7/10**	Good for operational data; needs SQL for payments
Scalability Reasoning	**8/10**	Geo matching & Kafka well justified; add estimations
Fault Tolerance	**5/10**	No circuit breakers, DLQ, or retry strategy discussed
Security	**6/10**	OTP good; missing auth token flow, data encryption at rest
Overall	**6.7/10**	**Strong foundation — add the gaps above for a**

Top 3 Things to Add Before Your Interview:

1. **Payment Service** — describe the Saga pattern, Aurora PostgreSQL, Stripe integration.
2. **Real-time location streaming** — WebSocket-based Location Stream Service fed by Kafka.
3. **Fault tolerance story** — circuit breakers, DLQ, Kafka retry, Redis cluster failover.